Asynchroniczny wybór lidera w anonimowym pierścieniu przy przybliżonej znajomości n

Raport do prezentacji: Itai, Rodeh, *Symmetry Breaking in Distributed Networks* (1990), §4.3.2

Stanisław Macura

listopad 2026

0. Zakres i źródła

Sekcja §4.3.2 to w artykule jeden akapit: algorytm z §3.4, w którym licznik weryfikacji przechodzi $2N-1$ krawędzi, a procesor, na którym się zatrzyma, „powiadamia pozostałych". Pozostałe elementy pochodzą z innych części artykułu: model (§3.1), selekcja (§3.4), ponawianie z numerami faz (Lemat 3.1), konieczność założenia $N \le n \le 2N-1$ (Tw. 4.1) i liczenie n przy znanym liderze (Lemat 4.1).

Moją inwencją jest przede wszystkim część techniczna, której artykuł nie opisuje: mechanizm powiadomienia (RESULT na kolejne $2N-1$ krawędzi) wraz z dowodem zgodności werdyktów, obsługa faz w asynchronii, terminacja przez COUNT+SIZE, wybór c przy nieznanym n, uproszczony dowód złożoności oraz implementacja w Go.

1 Postawienie problemu

Wariant. Rozważam dokładnie następujący model (artykuł §3.1):

- **pierścień jednokierunkowy** $v_0 \to v_1 \to \cdots \to v_{n-1} \to v_0$; każda krawędź ma **nieograniczony bufor FIFO**, kanały są niezawodne;
- **asynchroniczny**: brak wspólnego zegara; kolejność dostarczeń wyznacza *harmonogram* (adwersarz), w którym każda krawędź występuje nieskończenie często, więc każda wiadomość w końcu dociera;
- **anonimowy**: wszystkie procesory wykonują ten sam program, nie mają identyfikatorów; każdy ma **niezależne źródło losowości**;
- **start**: dowolny niepusty zbiór procesorów dostaje wiadomość budzącą (wybiera adwersarz); procesor śpiący budzi się na *dowolną* wiadomość;
- **wiedza**: znane jest N takie, że $N \le n \le 2N-1$; samo n jest nieznane.

Cel. Dokładnie jeden procesor zostaje *liderem*, każdy procesor v ma w rejestrze b_v wartość n, a wszystkie procesory **same wiedzą**, że skończyły (terminacja procesorów, nie tylko ustanie komunikacji). Algorytm ma być **Las Vegas**: wynik zawsze poprawny, kończy się z prawdopodobieństwem 1. Mierzymy oczekiwaną (po losowości) liczbę wiadomości i bitów w najgorszym przypadku po harmonogramach i zbiorach wake-upów.

Dlaczego akurat $N \le n \le 2N-1$. Tw. 4.1 artykułu: algorytm z terminacją procesorów, który nigdy się nie myli dla pierścienia rozmiaru m, musi się mylić dla rozmiaru $2m$. Dowód: sklejamy dwie kopie poprawnego wykonania na m procesorach; „bliźniaki" dostają te same rzuty i ten sam harmonogram, więc kończą z $b_v = m$ i dwoma liderami. Przedział $[N, 2N-1]$ jest *największym*

przedziałem postaci $[N, \cdot]$ bez pary $(m, 2m)$ — dopuszczenie $n = 2N$ już by ją wprowadziło. (Tw. 4.2 dodaje, że bez takiej wiedzy nie da się też mieć terminacji z ograniczonym błędem.)

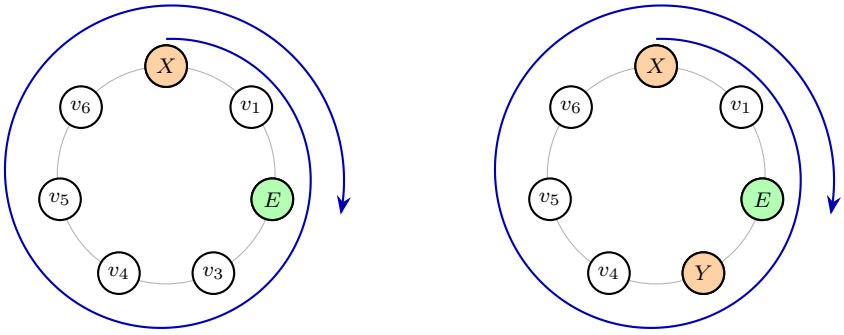
Co się psuje względem znanych wariantów. W §3.4 (async, n znane) kandydat wysyła licznik dookoła i rozpoznaje go po powrocie, bo licznik $= n$. Tu n nie znamy, więc **procesor nie rozpozna własnej wiadomości**. W §4.3.1 (sync, przybliżone n) informację niesie **cisza** w ustalonym oknie czasu; w asynchronii cisza nic nie znaczy. Obie przeszkody trzeba obejść jednocześnie.

2 Idea algorytmu i niezmienniki

Algorytm działa w *fazach* (ponowieniach) numerowanych $p = 0, 1, \ldots$; każda faza ma cztery etapy.

1. **Selekcja** (§3.4, bez zmian). Każdy aktywny procesor w każdej z c rund losuje bit, wysyła go BIT do następnego *aktywnego* (pasywni tylko przekazują) i odbiera bit poprzedniego aktywnego. Jeśli wylosował 0, a poprzednik 1 — staje się pasywny. Ocaleni po c rundach to *kandydaci*; $c = \lceil 2 \log_{4/3}(2N) \rceil$.
2. **Weryfikacja** (§4.3.2). Każdy kandydat wysyła VERIFY(d, k), gdzie d to liczba przebytych krawędzi, a k liczba napotkanych kandydatów (start liczy się jako 1, obcinamy do 3). Wiadomość zatrzymuje się po $2N - 1$ krawędziach; procesor E, na którym stanęła, wie, czy $k = 2$ (kandydat jedyny), czy $k \geq 3$.
3. **Powiadomienie**. E wysyła RESULT(u), $u = [k = 2]$, na kolejne $2N - 1$ krawędzi i sam od razu stosuje werdykt. Artykuł mówi tylko „this processor notifies the others".
4. **Wynik.** Werdykt „nie jedyny" $\Rightarrow$ procesor przechodzi do fazy $p + 1$ i zaczyna selekcję od nowa (Lemat 3.1). Werdykt „jedyny" $\Rightarrow$ kandydat zostaje liderem i wysyła COUNT(1); każdy inny zwiększa licznik i przekazuje; gdy licznik wraca do lidera, wynosi n (Lemat 4.1). Lider rozsyła SIZE(n); każdy zapisuje $b_v = n$, przekazuje i kończy.

Asynchronia. (a) *Start*: śpiący procesor budzi się na pierwszą otrzymaną wiadomość i wchodzi w rundę 1 — bity rundy 1 niosą pobudkę dookoła, więc w rundzie 1 aktywnych jest zawsze wszystkich n, niezależnie od zbioru wake-upów. (b) *Rundy są lokalne*: różne procesory mogą być w różnych rundach; numer rundy nie musi być przesyłany, bo pilnuje go FIFO (I2). (c) *Numery faz*: po ponowieniu w sieci krążą jeszcze VERIFY/RESULT starej fazy. Reguła: wiadomość z fazą mniejszą od mojej — odrzucam; z większą — „ktoś już wie, że trzeba zacząć od nowa", restartuję się w tej fazie i przetwarzam wiadomość w nowym stanie.



(a) jeden kandydat X: $k = 2$ (b) kandydaci X, Y: $k = 3$

Rysunek 1: $n = 7$, $N = 5$: trasa VERIFY ma $2N - 1 = 9$ krawędzi, kończy się w $E = v_2$. Trasa obejmuje odległość $n = 7$ (powrót przez X), ale nie $2n = 14$, więc mija X dokładnie raz.

Lemat 1 (trasa $2N-1$; fakt z §4.3)**.** *Jeśli $N \le n \le 2N-1$, to wiadomość wysłana z X i przekazywana przez $2N-1$ krawędzi odwiedza każdy procesor, a przez X przechodzi (nie licząc startu) dokładnie raz.*

Dowód. Trasa mija odległości $1, \ldots, 2N-1$ od X; X leży w odległościach podzielnych przez n. Z $n \le 2N-1$ odległość n jest na trasie (pełne okrążenie), a z $2n \ge 2N > 2N-1$ odległość $2n$ nie jest. □

Wniosek: przy jednym kandydacie licznik końcowy to $k = 2$ (start + powrót), przy $a \ge 2$ kandydatach $k \ge 3$ (start, inny kandydat, powrót). Tak należy czytać zdanie artykułu „passes through a leader at least three times" — „leader" oznacza tu kandydata, a start się wlicza.

Niezmienniki. W kodzie i dowodach utrzymywane są:

Niezmiennik I1 (§3.4)**.** W każdej fazie i rundzie jest ≥ 1 aktywny procesor; zatem po selekcji jest $a \ge 1$ kandydatów.

Niezmiennik I2 (FIFO)**.** Aktywny procesor w fazie p otrzymuje dokładnie jeden BIT na rundę, w kolejności rund — r-ty odebrany bit to bit rundy r poprzedniego aktywnego.

Niezmiennik I3 (koszt rundy)**.** W każdej rundzie każdą krawędzią przechodzi dokładnie jeden BIT; faza wysyła dokładnie cn bitów selekcji.

Niezmiennik I4 (separacja faz)**.** Na każdej krawędzi wszystkie BIT fazy p poprzedzają VERIFY/RESULT fazy p, a te poprzedzają wiadomości fazy $p+1$. W szczególności gdy VERIFY dociera do procesora, ten już wie, czy jest kandydatem.

Niezmiennik I5 (zgodność werdyktów)**.** Wszystkie werdykty RESULT jednej fazy są równe i prawdziwe.

Co utrzymujemy, co zmieniamy: I1–I4 są utrzymywane cały czas; liczba aktywnych maleje w oczekiwaniu czynnikiem $\frac{3}{4}$ na rundę; wiedza o wyniku rośnie: po weryfikacji zna go tylko E, po powiadomieniu — wszyscy.

3 Dowód poprawności

Krok 1 (I1, I2, art. §3.4). Odpaść może tylko procesor z 0, którego poprzedni aktywny wylosował 1 — a ten poprzednik w tej rundzie nie odpada. Jedyny aktywny dostaje własny bit, więc nie odpada. I2 przez indukcję po rundach: pasywni przekazują bez zmian, FIFO zachowuje kolejność; gdy poprzednik Q odpada w rundzie s, wysłał już bity $1..s$, a od tej chwili przekazuje bity $s{+}1, \ldots$ swojego poprzednika. I3: aktywni dzielą pierścień na odcinki, bit rundy przechodzi dokładnie swój odcinek. *Ważne*: z I2 wynika, że **wynik selekcji zależy tylko od rzutów, nie od harmonogramu** — dlatego analiza probabilistyczna działa dla najgorszego harmonogramu.

Krok 2 (I4). Procesor wysyła VERIFY fazy p dopiero po wysłaniu wszystkich swoich bitów fazy p, a pasywni przekazują w kolejności przyjścia; z FIFO bity fazy p wyprzedzają na każdej krawędzi VERIFY. Wiadomości fazy $p+1$ procesor wysyła dopiero po przetworzeniu RESULT(p) lub BIT($p{+}1$), a w `HandleMessage` najpierw przekazuje RESULT, potem wysyła nowy BIT — kolejność na krawędzi się zachowuje.

Lemat 2 (zgodność werdyktów, I5)**.** *W fazie z a kandydatami każdy werdykt jest „jedyny" wtedy i tylko wtedy, gdy $a = 1$.*

Dowód. Z I4 każdy procesor zna swój status, gdy liczy k. Dla $a = 1$ jest jedna trasa i $k = 2$ (Lemat 1). Dla $a \ge 2$ każda trasa obchodzi cały pierścień, więc mija innego kandydata oraz wraca przez nadawcę: $k \ge 3$. □

Krok 3: każdy opuszcza nieudaną fazę. Jeśli $a \geq 2$, RESULT(fałsz) z E ma pokryć E i kolejne $2N-1 \geq n-1$ procesorów, czyli wszystkich. Trasa może zostać ucięta tylko przez procesor, który już jest w fazie $p+1$ — ale on wysłał wtedy BIT(p+1), który restartuje następnika. Zatem każdy procesor przejdzie do fazy $p+1$ przez RESULT albo przez BIT nowej fazy.

Twierdzenie 1 (bezpieczeństwo — deterministyczne)**.** *W każdym wykonaniu (dla każdego harmonogramu i każdych rzutów) jest co najwyżej jeden lider, a jeśli algorytm się zakończy, to każdy $b_v = n$ i po zakończeniu w kanałach nie ma wiadomości.*

Szkic. Liderem zostaje tylko kandydat po werdykcie „jedyny", więc z Lematu 2 w tej fazie $a = 1$, a żaden werdykt nie jest „nie jedyny". Wiadomość fazy $p+1$ może powstać tylko z RESULT(fałsz) fazy p lub z innej wiadomości fazy $p+1$ — indukcyjnie nie powstaje żadna, więc faza jest ostatnia i lider jest jeden. Trasa RESULT pokrywa cały pierścień, a ewentualne powtórne kopie werdyktu (trasa $2N-1$ może dwukrotnie odwiedzić niektóre procesory, np. E) pomija flaga `ResultSeen`. COUNT startuje tylko lider i rośnie o 1 na krawędź, więc wraca z wartością n (art. Lemat 4.1); SIZE niesie ją do wszystkich. Pusty stan kanałów: procesor wysyła SIZE jako ostatnią wiadomość i się zatrzymuje, a wszystko, co wcześniej przyszło na jego krawędź wejściową, wyprzedza SIZE (FIFO); indukcyjnie po pierścieniu za SIZE nic nie płynie. Lider zatrzymuje się ostatni, gdy SIZE do niego wraca. □

Twierdzenie 2 (żywotność)**.** *Faza kończy się sukcesem z prawdopodobieństwem $> 1 - 1/n$, niezależnie od poprzednich faz; liczba faz F ma rozkład geometryczny, algorytm kończy się z prawdopodobieństwem 1 i $\mathbb{E}[F] < \frac{1}{1-1/n} \leq 2$.*

Dowód — rachunek z sekcji 4; struktura „faza udaje się z $p > 0 \Rightarrow$ ponowienia kończą się z prawd. 1" to Lemat 3.1 artykułu. Zauważmy: Tw. 4.1 nie jest naruszone, bo algorytm używa N, a w $[N, 2N-1]$ nie ma pary $(m, 2m)$.

Mapa do artykułu. Krok 1 ↔ §3.4 (akapit pod opisem selekcji); Lemat 1 ↔ wstęp §4.3; Lemat 2 ↔ nawias „at least three times" w §4.3.2; ponowienia ↔ dowód Lematu 3.1; liczenie n ↔ Lemat 4.1; konieczność założenia ↔ Tw. 4.1–4.2.

4 Złożoność

Ile zostaje kandydatów. Niech m_t — liczba aktywnych po t rundach ($m_0 = n$), $Y_t = m_t - 1 \geq 0$. Gdy $m \geq 2$, poprzedni aktywny jest innym procesorem, więc każdy aktywny odpada z prawdopodobieństwem dokładnie $\frac{1}{4}$: $\mathbb{E}[m_t \mid m_{t-1} = m] = \frac{3}{4}m$, czyli $\mathbb{E}[Y_t \mid Y_{t-1}] \leq \frac{3}{4}Y_{t-1}$ (dla $m = 1$: Y zostaje 0). Stąd $\mathbb{E}[Y_c] \leq (n-1)(\frac{3}{4})^c$ i z nierówności Markowa

$$\Pr[a \geq 2] = \Pr[Y_c \geq 1] \leq (n-1)\left(\tfrac{3}{4}\right)^c.$$

Artykuł dochodzi do tego samego rzędu przez pełny rozkład i wariancję (Lematy 3.2–3.3, analiza Hofriego, Czebyszew); powyższe trzy linijki wystarczają do tezy.

Wybór c. W §3.4 jest $c = 5\log n$, ale n jest nieznane — zastępuję je przez $2N > n$: $c = \lceil 2\log_{4/3}(2N) \rceil$ daje $(\frac{3}{4})^c \leq (2N)^{-2} < n^{-2}$, więc $\Pr[a \geq 2] < 1/n$ oraz $\mathbb{E}[a] \leq 1 + \mathbb{E}[Y_c] < 1 + 1/n$. Ponadto c **nie może być zbyt małe**: odpadający procesor ma ocalałego poprzednika (z bitem 1), więc w rundzie odpada co najwyżej połowa aktywnych; przy $2^c < n$ zawsze zostaje ≥ 2 kandydatów i algorytm ponawia w nieskończoność. Program odrzuca takie c.

Suma. Fazy są niezależne (świeże rzuty), więc z tożsamości Walda

$$\mathbb{E}[\#\text{wiad.}] \leq \mathbb{E}[F]\big(cn + 2\,\mathbb{E}[a](2N-1)\big) + 2n \leq 2\big(cn + 2(1+\tfrac{1}{n})(2N-1)\big) + 2n = O(n\log n),$$

bo $c = O(\log N) = O(\log n)$ i $N \leq n$. Bity: selekcja $cn \cdot O(1)$, reszta $O(n)$ wiadomości po $O(\log N)$ bitów, razem $O(n \log n)$ — tak jak w artykule. Numer fazy dokłada $O(\log F)$ bitów do wiadomości, a $\mathbb{E}[F \log F] = O(1)$, więc asymptotyka się nie zmienia.

etap	wiadomości na fazę	bitów na wiadomość	źródło kosztu
1. selekcja	cn (dokładnie, I3)	$O(1)$	art. §3.4
2. weryfikacja	$\leq a(2N-1)$	$O(\log N)$	art. §4.3.2
3. powiadomienie	$\leq a(2N-1)$	$O(\log N)$	własne
4. liczenie n (raz)	$2n$	$O(\log n)$	art. Lemat 4.1 + własne

Tabela 1: Koszt jednej fazy; a — liczba kandydatów. „$\leq$", bo w nieudanej fazie trasy mogą zostać ucięte.

Uwagi. (1) Zdanie artykułu „$O(n)$ message" jest skrótem: selekcja to $\Theta(n \log n)$ wiadomości *jednobitowych*; $O(n)$ dotyczy wiadomości weryfikacji/powiadomienia/liczenia (po $O(\log N)$ bitów). Dlatego w tabeli 2 wiadomości też rosną jak $n \log n$. (2) Dolne ograniczenie $\Omega(n \log n)$ (crossing sequences, [MZ]) artykuł podaje dla algorytmów deterministycznych — tu tylko je cytuję. (3) Artykuł nie analizuje czasu; przy jednostkowym opóźnieniu wiadomości selekcja trwa $O(cn)$, a weryfikacja i powiadomienie $O(N)$.

5 Implementacja (Go)

Cały program to `kod/main.go`. Najważniejsze elementy:

- `NodeState` — stan jednego procesora jako osobna struktura: `Phase`, `Role` (`Sleep`, `Active`, `Passive`, `Candidate`, `Leader`, `Knows`), `Round`, `MyBit`, `IsCand`, `ResultSeen`, `KnownN`, `Halted` oraz własny generator losowy. `ID` służy tylko do logów — algorytm go nie czyta.
- `Message` — jeden typ dla wszystkich wiadomości (`Wake`, `Bit`, `Verify`, `Result`, `Count`, `Size`) z polami `Phase`, `B`, `D`, `K`, `U`, `J`; `Round` jest przesyłane wyłącznie dla czytelności logu.
- `HandleMessage(m, cfg)` — **kod obsługi jednej wiadomości** (wymóg dla sieci asynchronicznej): budzenie/restart przy nowszej fazie, odrzucenie przeterminowanej, potem `switch` po typie. Gałąź `Verify` na końcu trasy tworzy `Result` z `D: -1` i przez `fallthrough` od razu stosuje werdykt w tym samym procesorze — to jest realizacja kroku 3. Funkcja zwraca listę wiadomości do następnika.
- `node` — gorutyna procesora: czyta z kanału `in` (bez bufora), woła `HandleMessage`, oddaje listę wysłanych na kanał `out`; kończy się po `Halted`.
- `run` — **dyspozytor** w `main`: trzyma kolejki FIFO krawędzi (`queues[j]` to krawędź $v_{j-1} \to v_j$), w pętli `pickNonEmpty` losuje niepustą kolejkę (losowy harmonogram, powtarzalny przez `-seed`), dostarcza jedną wiadomość, dopisuje odpowiedzi do kolejki następnika, loguje „`vX wysłał M do vY`" i czeka globalne `Delay`. Kończy, gdy wszystkie kolejki są puste.
- `wakeSet` — zbiór wake-upów (`one`/`all`/`rand`); `Stats.Count` — liczniki wiadomości wg typu i bitów (BIT: 1, VERIFY: $\lceil \log_2 2N \rceil + 2$, RESULT: $\lceil \log_2 2N \rceil + 1$, COUNT/SIZE: $\lceil \log_2 2N \rceil$; znaczniki typu i fazy pominięte jako $O(1)$ w oczekiwaniu); `summary` — podsumowanie i kontrola poprawności (1 lider, wszyscy znają n, wszyscy zakończyli; inaczej kod wyjścia 1).

Dlaczego dyspozytor, a nie kanały między gorutynami. Przy kanałach bez bufora łączących sąsiadów w cyklu procesor blokuje się na wysyłaniu do następnika, który sam blokuje się na wysyłaniu — zakleszczenie. Model z artykułu ma nieograniczone bufory, więc trzymam je jawnie w dyspozytorze; kanały gorutyn pozostają bez bufora. W każdej chwili dostarczana jest jedna wiadomość, co odpowiada modelowi §3.1 („at any time at most one message arrives").

Odstępstwo od modelu. §3.1 pozwala wysłać po zdarzeniu co najwyżej jedną wiadomość; u mnie koniec trasy wysyła RESULT i bit nowej fazy, a lider — RESULT i COUNT. Da się to zasymulować lokalną kolejką wyjściową; nie zmienia to poprawności ani złożoności.

6 Wyniki eksperymentów

Tabela 2: średnie z 20 uruchomień (`-seed 1..20`, `-wake rand`, domyślne c) dla dwóch skrajnych wartości N: $N = n$ oraz $n = 2N - 1$. We wszystkich 320 uruchomieniach wynik był poprawny. Tabelę generuje skrypt `raport/bench.sh`.

n	N	c	BIT	VERIFY	RESULT	C+S	wiad.	bity	fazy	$\frac{\text{wiad.}}{n \log_2 n}$	$\frac{\text{bity}}{n \log_2 n}$	OK
8	8	20	160	15	15	16	206	389	1.00	8.58	16.21	20/20
16	16	25	400	31	31	32	494	963	1.00	7.72	15.05	20/20
32	32	29	928	63	63	64	1118	2257	1.00	6.99	14.11	20/20
64	64	34	2176	127	127	128	2558	5231	1.00	6.66	13.62	20/20
128	128	39	4992	255	255	256	5758	11885	1.00	6.43	13.26	20/20
256	256	44	11264	511	511	512	12798	26603	1.00	6.25	12.99	20/20
512	512	49	25088	1023	1023	1024	28158	58857	1.00	6.11	12.77	20/20
1024	1024	54	55296	2047	2047	2048	61438	128999	1.00	6.00	12.60	20/20
8	5	17	136	9	9	16	170	299	1.00	7.08	12.46	20/20
16	9	21	336	17	17	32	402	717	1.00	6.28	11.20	20/20
32	17	25	800	33	33	64	930	1679	1.00	5.81	10.49	20/20
64	33	30	1920	65	65	128	2178	3921	1.00	5.67	10.21	20/20
128	65	34	4352	129	129	256	4866	8851	1.00	5.43	9.88	20/20
256	129	39	9984	257	257	512	11010	19989	1.00	5.38	9.76	20/20
512	257	44	22528	513	513	1024	24578	44567	1.00	5.33	9.67	20/20
1024	513	49	50176	1025	1025	2048	54274	98329	1.00	5.30	9.60	20/20

Tabela 2: Skalowanie. Górna część: $N = n$; dolna: $N = \lceil (n+1)/2 \rceil$, czyli $n = 2N - 1$ (C+S = COUNT + SIZE).

Obserwacje, zgodne z tabelą 1:

- BIT $= c \cdot n \cdot F$ dokładnie (I3), np. $54 \cdot 1024 = 55296$; VERIFY = RESULT $= 2N - 1$ (zawsze $a = 1$ i $F = 1$ przy domyślnym c), C+S $= 2n$.
- Iloraz wiadomości do $n \log_2 n$ maleje i stabilizuje się: dominuje selekcja z $c / \log_2 n \to 2 / \log_2 \frac{4}{3} \approx 4{,}82$; przy $N = n$ jest nieco więcej, bo c liczone jest od $2N = 2n$.
- Przy $n = 2N - 1$ wiadomości jest mniej: c jest mniejsze, a trasy $2N - 1 \approx n$ zamiast $\approx 2n$.

Tabela 3 pokazuje kompromis przy zmianie c ($n = N = 64$, 20 seedów). Za małe c (10–11, blisko granicy $2^c \geq n$) daje dziesiątki–setki ponowień i wielokrotnie większy koszt; domyślne $c = 34$ daje zawsze jedną fazę, ale płaci za ostrożność w selekcji. Minimum kosztu leży w okolicy $c \approx 14$–17, czyli $\approx \log_{4/3} n$ — zgodnie z intuicją z §3.4.

c	śr. fazy	maks. fazy	BIT	VERIFY+RESULT	wiadomości	bity
10	128.25	519	82080	66520	148728	673381
11	18.30	44	12883	7578	20589	80746
12	3.30	4	2534	1100	3762	13054
14	2.10	6	1882	604	2613	8014
17	1.90	3	2067	540	2735	7640
20	1.85	2	2368	524	3020	7801
34	1.00	1	2176	254	2558	5231

Tabela 3: Wpływ liczby rund selekcji c ($n = N = 64$, średnie z 20 uruchomień).

7 Jak uruchomić

Wymagany Go $\geq$ 1.21 (kod używa wbudowanego `min`). Na Ubuntu: `sudo apt install golang-go` (Ubuntu $\geq$ 24.04; na 22.04 pakiet ma Go 1.18 — wtedy `sudo snap install go --classic`).

```
cd kod
go run . -n 6 -N 4 -c 3 -wake one -seed 52 -delay 200ms   # demo z ponowieniem
go run . -n 1024 -N 600 -wake rand -quiet                 # tylko podsumowanie
```

Flagi: `-n` rzeczywisty rozmiar; `-N` znane N (wymagane $N \le n \le 2N-1$); `-c` liczba rund selekcji ($0 =$ domyślne $\lceil 2\log_{4/3} 2N \rceil$; odrzucane, gdy $2^c < n$); `-wake one|all|rand`; `-seed` ziarno (harmonogram i rzuty — przebieg jest powtarzalny); `-delay` globalne opóźnienie między dostarczeniami (domyślnie 10 ms, `0` = bez opóźnienia); `-quiet` bez logu wiadomości. Program kończy się kodem 1, jeśli wynik jest niepoprawny.

Przebieg `-seed 52` z pierwszego polecenia: w fazie 0 są 4 kandydaci, wszystkie trasy kończą się z $k = 3$, a RESULT(fałsz) restartuje pierścień (widać ucinanie tras z kroku 3: zamiast $4 \cdot 7$ wysłano tylko 6 wiadomości RESULT fazy 0, resztę pierścienia zrestartowały bity fazy 1); w fazie 1 zostaje jeden kandydat, który zostaje liderem i ustala $n = 6$; podsumowanie: BIT $36 = 3 \cdot 6 \cdot 2$, VERIFY 35, RESULT 13, COUNT 6, SIZE 6.

Całość (program, tabele, ten raport) buduje skrypt w katalogu głównym:

```
./build.sh deps      # jednorazowo: pakiety apt (Go, TeX Live z polskim)
./build.sh           # kompilacja programu + raport/raport.pdf
./build.sh bench     # ponowne pomiary -> raport/tabela*.tex, potem raport
```